23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Lost and Found Web App

Group No: A10

Team Members

Roll No	Student Name
AM.SC.U4CSE25015	Adithyanandan A
AM.SC.U4CSE25035	Ganesh S
AM.SC.U4CSE25045	Muhammad Yazeen
AM.SC.U4CSE25046	Rahul R Nair

Contents

No.	Chapter	Page
1	Abstract	3
2	Introduction	3
3	Requirements (Functional and Non-functional)	5
4	UI Design	7
5	Database Design	9
6	Database Selection and Justification	14
7	Implementation	15
8	Testing and Validation	17
9	Results and Screenshots	19
10	Challenges Faced and Solutions	20
11	Conclusion and Future Enhancements	21
	References	22

The live listing counts shown later in the report are a snapshot and may change.

1. Abstract

The Lost and Found Web App provides a shared online place for a community to report and search for lost or found belongings. The application offers public lost-item and found-item listings with search and filters, while signed-in users can create and manage their own posts. It is implemented as a static website hosted on GitHub Pages and connects from the browser to Supabase for authentication, a PostgreSQL database, and image storage. The relational database separates user profiles, items, categories, locations, images, and contact messages, and uses primary keys, foreign keys, enumerated status values, validation constraints, indexes, triggers, and row-level security policies to support integrity and controlled access. The schema also contains tables planned for future claims and messaging workflows. In the current demo, a signed-in Adithyan account has an existing lost-item report, and visitors can browse the live listings. The project demonstrates how a browser-based application can use a managed relational database to store and retrieve connected application data.

2. Introduction

2.1 Background

People lose belongings in classrooms, libraries, parks, transport areas, and other shared spaces. A person who finds an item may not know its owner, while the owner may not know where to ask. In many communities, reports are distributed across informal messages, notice boards, group chats, or personal conversations. These approaches are difficult to search, may be visible only to a limited group, and do not provide a consistent way to update a report when an item is claimed or returned.

2.2 Problem statement

The project addresses the lack of a central, searchable record for lost and found items. A useful system needs to store enough details to identify a report, such as its title, description, category, location, date, status, owner, and optional image. It must also distinguish public browsing from account-specific changes: visitors should be able to find listings, while a user should manage only the posts they own.

2.3 Proposed solution

The proposed solution is a responsive web application with separate Lost Items and Found Items pages. Users can search by text and narrow listings by category or status. An authenticated user can post an item, optionally upload a photo, review their posts on a dashboard, edit or delete a post, and change its status. A contact form stores messages for follow-up. The interface is served as static HTML, CSS, and JavaScript from GitHub Pages. Supabase provides the hosted PostgreSQL database, authentication, an API used by the browser, and object storage for item images.

2.4 Objectives

- Provide a clear public catalogue of lost and found reports.
- Allow a visitor to search and filter reports without signing in.
- Require authentication for creating and managing a user's own reports.
- Represent users, items, categories, locations, and images as related database records.
- Enforce data integrity and ownership rules in PostgreSQL rather than relying only on interface behavior.
- Deploy the frontend without maintaining a separate application server.

2.5 Scope and limitations

The current application implements account sign-up, sign-in, sign-out and password reset; public item browsing and client-side search/filtering; authenticated item creation, editing, deletion and status updates; image upload; a user dashboard; and contact-message submission. The database schema also defines claims, evidence, conversations, messages, notifications, tags, roles, and status history. Those tables are schema groundwork: the current interface does not yet implement those workflows. The application does not currently include automated matching, in-app claim review, private messaging, moderation screens, or a report-generation module. Existing Firebase accounts and data were not transferred; the current demo uses Supabase data.

3. Requirements (Functional and Non-functional)

3.1 Functional requirements

ID	Requirement	Current implementation
F1	Display lost and found reports to visitors.	Separate public Lost Items and Found Items pages read the corresponding item kind from Supabase.
F2	Find reports by text and narrow results by category, status, and order.	The listing page filters the loaded records in the browser by title, description, location, category, and status, and supports newest/oldest ordering.
F3	Register, sign in, sign out, and request a password reset.	Supabase Auth email/password methods are called by the frontend.
F4	Create a lost or found report.	An authenticated user submits title, description, category, location, date, contact details, and an optional image.
F5	Manage an account's own reports.	The dashboard lists the signed-in user's reports and supports editing, deletion, and status changes.
F6	Store and display an optional item image.	The image file is uploaded to the `item-images` Supabase Storage bucket; its path and public URL are recorded in `item_images`.
F7	Accept a contact message.	The contact form inserts the sender details and message into `contact_messages`.
F8	Associate each item with a valid owner and category, and optionally a location.	Foreign keys and non-null rules enforce the required associations.

3.2 Non-functional requirements

ID	Quality	Requirement and design response
N1	Usability	Use consistent navigation, distinct lost/found sections, readable item cards, and visible search and filter controls.
N2	Security	Use Supabase Auth for identity and PostgreSQL row-level security for owner-scoped writes. The browser uses a public project key; a service-role key must not be placed in static files.
N3	Integrity	Use primary and foreign keys, NOT NULL, UNIQUE, CHECK constraints, and enums to reject invalid or orphaned data.
N4	Performance	Create indexes for common item browsing and owner lookups, and trigram indexes for

ID	Quality	Requirement and design response
		title/description search. The current search is client-side; server-side search and pagination are future improvements.
N5	Availability	Host the static frontend on GitHub Pages and use Supabase-managed services. The application depends on network access and availability of both platforms.
N6	Maintainability	Keep schema creation in a versioned SQL migration and separate page logic into JavaScript modules for authentication, posts, search, dashboard, and contact.
N7	Scalability	Relational keys and indexes provide a foundation for growth. Large datasets will require server-side filtering, pagination, and query-plan review.
N8	Compatibility	Use standard browser technologies and Supabase's JavaScript client so the frontend can be served as static files.

4. UI Design

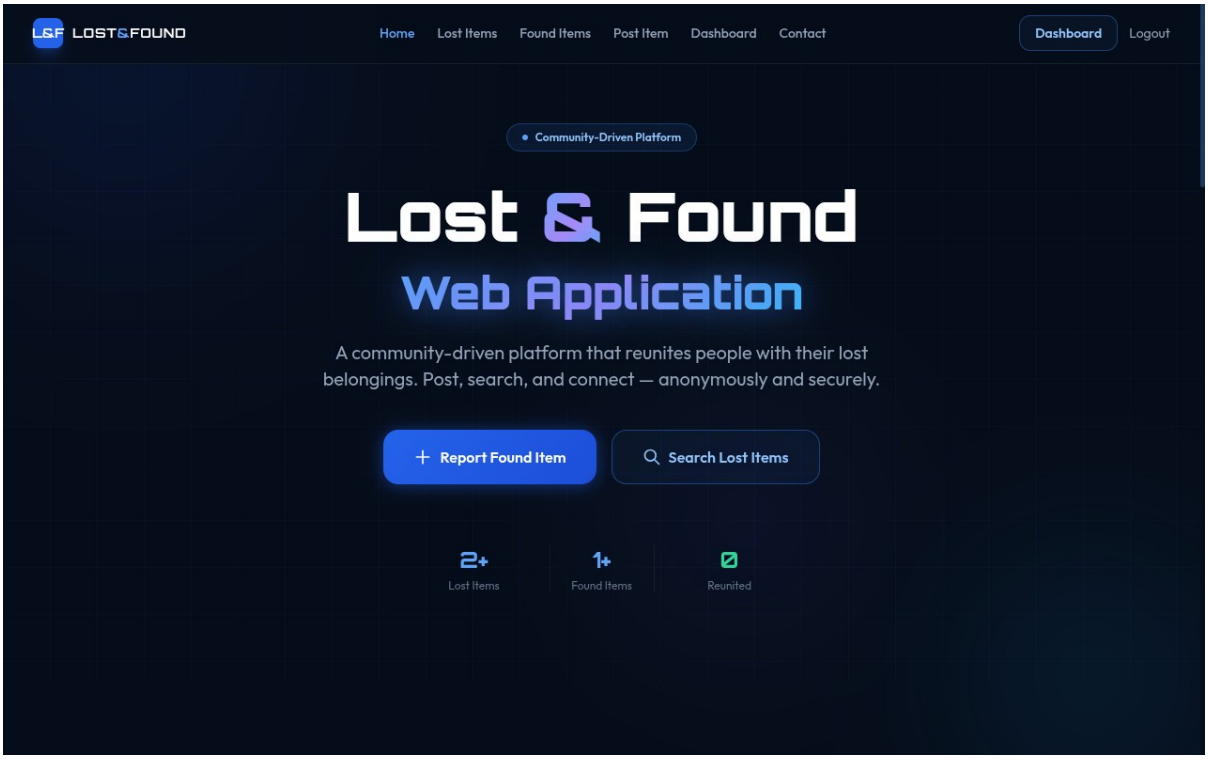
4.1 Navigation and user flow

The global navigation links to Home, Lost Items, Found Items, Post Item, Dashboard, and Contact. A visitor can start at Home, browse a listing, and use search or filters. Selecting Post Item leads to authentication when no session exists. After signing in, the user can submit a report and manage it from the dashboard. The dashboard separates the user's lost and found posts and offers edit, delete, and status controls for those posts.

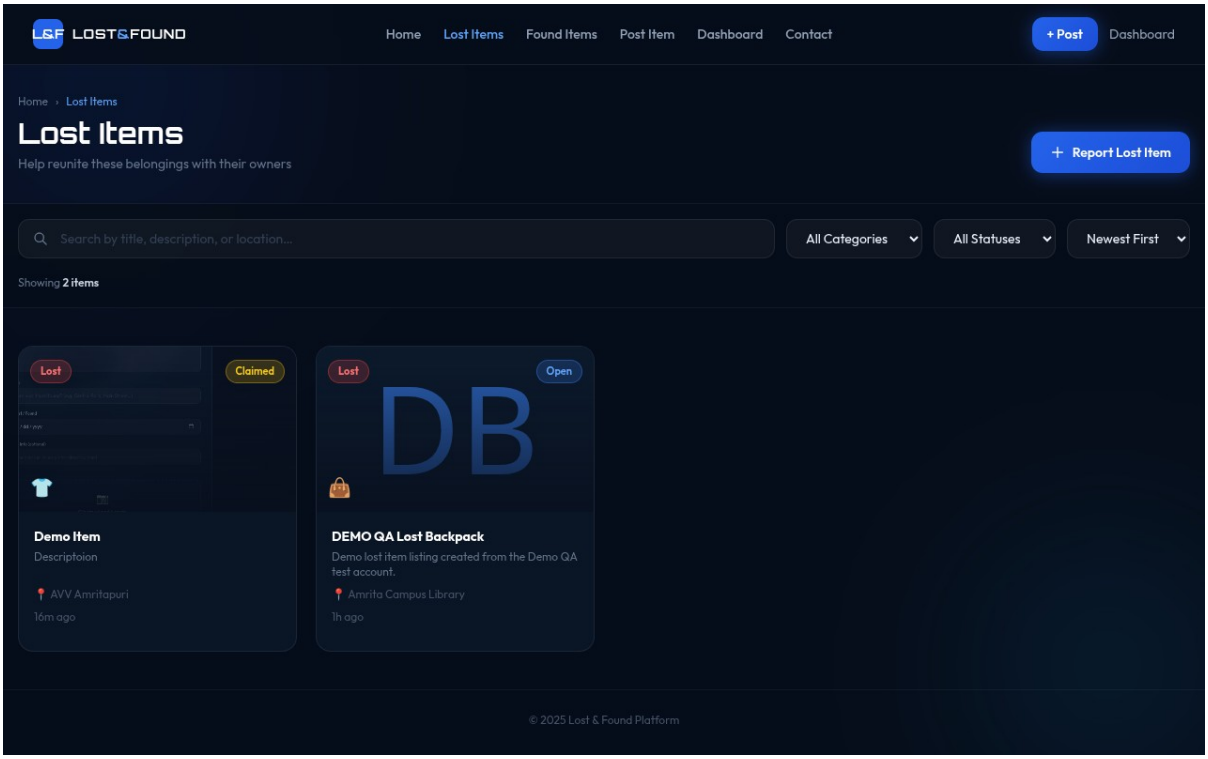
> **Typical flow:** Home → Lost Items or Found Items → Search/filter → inspect a listing. For a contributor: Sign in → Post Item → submit details/photo → Dashboard → edit, update status, or delete the user's post.

4.2 Main screens

Screen	Purpose and user interaction
Home	Introduces the service and offers shortcuts to report a found item, search lost items, and navigate to the main sections. When signed in, account navigation provides dashboard access and sign-out.
Lost Items / Found Items	Shows cards for the selected report type. Each card can show title, description, category, location, status, date, and photo/fallback image. Search, category, status, and sorting controls narrow the visible cards.
Sign Up / Sign In / Password Reset	Provides account access using Supabase Auth. A signed-in identity is linked to an application profile.
Post Item	Collects whether the report is lost or found plus the item's details. A user may select an image, which is uploaded to Storage and linked to the database record.
Dashboard	Shows account profile information and the user's lost/found posts. The owner can edit, delete, or change a post's status.
Contact	Collects a name, email, subject, and message and stores the submission in the database.



Home screen in the signed-in demo state, with the main navigation and report entry points visible.

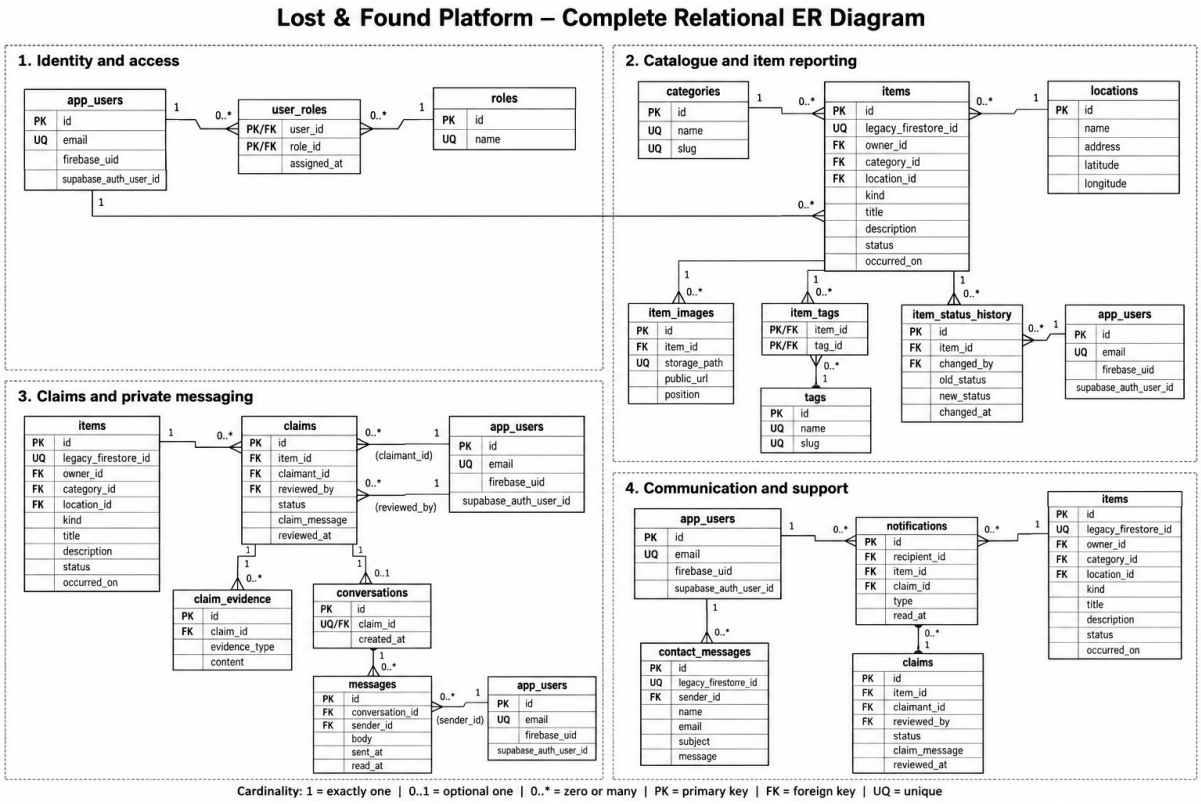


Lost Items screen with the existing demo lost item, another listing, and search/filter controls.

5. Database Design

5.1 ER diagram and model-to-relational conversion

The database is represented by the ER diagram below. A strong entity becomes a table with a primary key. A one-to-many relationship is represented by placing a foreign key on the many side. Many-to-many relationships are resolved using bridge tables. For example, users and roles are connected through `user_roles`, and items and tags through `item_tags`. Optional relationships use nullable foreign keys, such as an item's location or a contact message's sender.



Complete relational ER diagram from the project's db folder. Repeated entity boxes are shown in more than one area to keep related parts of the diagram readable.

5.2 Entities and relational schema

The migration defines the following relations. UUID identifiers are generated for most records; small lookup tables use small integer identities. PK means primary key, FK means foreign key, and UQ means unique.

Relation	Important attributes and keys	Role in the system
`app_users`	`id UUID PK`; `firebase_uid TEXT UQ`; `supabase_auth_user_id UUID UQ`; `email CITEXT UQ`; display_name TEXT NOT NULL; phone TEXT?; avatar_url TEXT?; created_at TIMESTAMPTZ; migrated_at TIMESTAMPTZ?.	Profile linked to Supabase Auth; legacy identity fields are optional.

Relation	Important attributes and keys	Role in the system
roles	id SMALLINT PK identity; name TEXT UQ NOT NULL; description TEXT?.	Role lookup: user, moderator, admin.
user_roles	user_id UUID PK/FK → app_users; role_id SMALLINT PK/FK → roles; assigned_at TIMESTAMPTZ.	Profile-role bridge; composite primary key.
categories	id SMALLINT PK identity; name TEXT UQ NOT NULL; slug TEXT UQ NOT NULL; icon TEXT?; active BOOLEAN NOT NULL DEFAULT true.	Reusable category lookup.
locations	id UUID PK; name TEXT NOT NULL; address TEXT?; latitude/longitude NUMERIC(9,6)?; created_at TIMESTAMPTZ. Coordinates must be paired and in range.	Reusable location details.
items	id UUID PK; legacy_firestore_id TEXT UQ?; owner_id UUID FK NOT NULL → app_users; category_id SMALLINT FK NOT NULL → categories; location_id UUID FK? → locations; kind ITEM_KIND NOT NULL; title TEXT NOT NULL; description TEXT NOT NULL; occurred_on DATE?; contact TEXT?; status ITEM_STATUS NOT NULL DEFAULT open; created_at/updated_at TIMESTAMPTZ.	Main lost/found report. Owner/category deletes are restricted; location deletion sets the optional FK to NULL.
item_images	id UUID PK; item_id UUID FK NOT NULL → items; storage_path TEXT UQ NOT NULL; public_url TEXT?; alt_text TEXT?; position SMALLINT NOT NULL DEFAULT 1; uploaded_at TIMESTAMPTZ; UQ(item_id, position).	Image metadata; item deletion cascades.
tags	id SMALLINT PK identity; name TEXT UQ NOT NULL; slug TEXT UQ NOT NULL.	Reusable tag lookup.
item_tags	item_id UUID PK/FK → items; tag_id SMALLINT PK/FK → tags.	Item-tag bridge; both FK deletions cascade.
item_status_history	id BIGINT PK identity; item_id UUID FK NOT NULL → items; changed_by UUID FK? → app_users; old_status ITEM_STATUS?; new_status ITEM_STATUS NOT NULL; note TEXT?; changed_at TIMESTAMPTZ.	Status audit history schema; no current UI workflow writes it.
claims	id UUID PK; item_id UUID FK NOT NULL → items; claimant_id UUID FK NOT NULL → app_users; status CLAIM_STATUS NOT NULL DEFAULT pending; claim_message TEXT NOT NULL; reviewed_by UUID FK?; reviewed_at TIMESTAMPTZ?; created_at/updated_at TIMESTAMPTZ; review-pair and claimant/reviewer checks.	Future claim workflow. Partial unique index allows one pending/approved claim per item and claimant.
claim_evidence	id UUID PK; claim_id UUID FK NOT NULL → claims; evidence_type TEXT CHECK(photo/document/description);	Evidence associated with a claim; claim deletion

Relation	Important attributes and keys	Role in the system
	content TEXT NOT NULL; submitted_at TIMESTAMPTZ.	cascades.
`conversations`	`id UUID PK`; `claim_id UUID FK UQ NOT NULL` → claims; created_at TIMESTAMPTZ.	At most one conversation per claim.
`messages`	`id UUID PK`; `conversation_id UUID FK NOT NULL` → conversations; `sender_id UUID FK NOT NULL` → app_users; body TEXT NOT NULL; sent_at TIMESTAMPTZ; read_at TIMESTAMPTZ?.	Future private messaging; conversation deletion cascades.
`notifications`	`id UUID PK`; `recipient_id UUID FK NOT NULL` → app_users; item_id UUID FK?; claim_id UUID FK?; type NOTIFICATION_TYPE NOT NULL; body TEXT NOT NULL; read_at TIMESTAMPTZ?; created_at TIMESTAMPTZ.	Future notification records.
`contact_messages`	`id UUID PK`; legacy_firestore_id TEXT UQ?; `sender_id UUID FK?` → app_users; name TEXT NOT NULL; email CITEXT NOT NULL; subject TEXT NOT NULL; message TEXT NOT NULL; created_at TIMESTAMPTZ.	Contact form messages; account sender is optional.

5.3 Relationships and cardinalities

Relationship	Cardinality	Implementation
Profile to item	One profile to zero or many items; each item has exactly one owner.	`items.owner_id` is a required FK.
Category to item	One category to zero or many items; each item has one category.	`items.category_id` is a required FK.
Location to item	One location to zero or many items; an item may have no location.	Nullable `items.location_id` FK.
Item to image	One item to zero or many image records; each image belongs to one item.	`item_images.item_id` FK, with cascade delete.
Profile to role	Many-to-many.	`user_roles` bridge table.
Item to tag	Many-to-many.	`item_tags` bridge table.
Item to claim	One item to zero or many claims; each claim refers to one item and one claimant.	Foreign keys in `claims`; a partial unique index permits only one active claim per item/user pair.
Claim to evidence	One claim to zero or many evidence	`claim_evidence.claim_id` FK.

Relationship	Cardinality	Implementation
	records.	
Claim to conversation	One claim to zero or one conversation.	Unique `conversations.claim_id` FK.
Conversation to message	One conversation to zero or many messages.	`messages.conversation_id` FK.
Profile to contact message	One profile may submit many messages; a message may have no account sender.	Nullable `contact_messages.sender_id` FK.

5.4 Constraints, keys, and indexes

- **Identity and uniqueness:** UUID primary keys use `gen_random_uuid()`. Email is stored as `citext` and unique in the profile table. Category and tag names/slugs and image storage paths are unique.
- **Domain restrictions:** PostgreSQL enums constrain item kind to `lost`/`found`, item status to `open`/`claimed`/`reunited`/`archived`, claim status to its defined review states, and notification type to supported categories.
- **Validation:** Checks enforce non-empty bounded titles/descriptions/names/messages, valid lowercase slugs, positive image positions, coordinate ranges, and consistent claim review fields. A claim cannot be reviewed by its claimant.
- **Referential integrity:** Foreign keys prevent orphaned items and related records. Delete actions use RESTRICT for ownership/category records, CASCADE for dependent image/tag/claim data, and SET NULL where an optional association may be removed.
- **Indexes:** Item browsing is indexed by kind/status/creation time; owner, category, and location lookups have indexes. GIN trigram indexes cover item title and description. Additional indexes support claims, messages, notifications, and status-history lookups.

5.5 Functional dependencies and candidate keys

A functional dependency X → Y means that a value of X determines exactly one value of Y in a relation. The UUID primary key of each entity is a guaranteed candidate key. For example, `items.id → owner_id, category_id, location_id, kind, title, description, occurred_on, contact, status, created_at, updated_at`. The closure $\{items.id\}^+$ contains all attributes of `items`, so `items.id` is a candidate key and, by the schema's primary-key declaration, the chosen primary key.

For lookup and bridge tables, additional dependencies include `categories.id → name, slug, icon, active`; `categories.slug → id, name, icon, active`; `item_images.id → item_id, storage_path, public_url, alt_text, position, uploaded_at`; `(item_id, position) → image attributes`; `(item_id, tag_id) → no additional non-key attributes`; and `(user_id, role_id) → assigned_at`. The schema also declares unique names/slugs for categories and tags, unique role names, and a unique claim reference for conversations. Nullable unique legacy identifiers are treated as integrity constraints for supplied values, not as guaranteed candidate keys for every row.

5.6 Normalization

First normal form (1NF): Each column stores a single value. A report has one category and an optional location reference; repeated photos and tags are not stored as comma-separated values. They are represented as separate rows in `item_images` and `item_tags`.

Second normal form (2NF): Tables with composite primary keys contain no non-key attribute that depends on only part of the key. The `user_roles` and `item_tags` tables are pure association tables. Their relationship is identified by the complete pair of foreign keys.

Third normal form (3NF): Descriptive user, category, and location data are stored in their own relations rather than repeated in each item row. In `items`, the category identifier references `categories`; category name and icon are not duplicated in every item. The same applies to owner profile and location data. This reduces update, insertion, and deletion anomalies.

BCNF assessment: Under the declared keys and unique constraints, each non-trivial determinant in the principal relations is a candidate key (or the declared key). The bridge relations have only key attributes or attributes dependent on the full composite key. The schema therefore satisfies BCNF for these relations based on the specified functional dependencies. The partial uniqueness rule for active claims applies only to a subset of rows and is an additional business rule rather than a global candidate key.

6. Database Selection and Justification

PostgreSQL was selected as the database system and is hosted through Supabase. This choice fits the application because its data has clear relationships: each item belongs to a user and category, can reference a location, and may have associated images. Relational constraints make those links explicit and protect data consistency when the browser submits changes.

6.1 Reasons for selecting PostgreSQL

- **Relational model and SQL:** Primary and foreign keys, joins, unique constraints, and check constraints directly represent the project's entities and business rules.
- **Strong integrity features:** Enums, transactional writes, triggers, and constraint checks help keep item types, statuses, and associations valid.
- **Search and indexing options:** B-tree indexes support common browse and owner lookups; the `pg_trgm` extension supports trigram indexes on titles and descriptions.
- **Supabase integration:** Supabase provides a hosted Postgres database, an API and JavaScript client, authentication, object storage, and row-level security policies that can be evaluated for browser requests.
- **Static deployment fit:** The browser can call Supabase directly using the project's public key while PostgreSQL RLS checks which rows the authenticated user may change. This avoids maintaining a separate custom application server for the current scope.
- **Cost and development:** The project uses managed cloud services and standard SQL tooling. The actual service limits and pricing depend on the Supabase plan and should be checked in the project dashboard.

6.2 Why this model fits the application

The item catalogue is structured and benefits from referential integrity: a report should not point to a nonexistent owner or category. A relational database also makes it straightforward to retrieve an item together with its category, location, owner reference, and image metadata. Supabase keeps the database and related authentication/storage services under one project, which simplifies the demo deployment. The design still keeps image bytes in object storage rather than inside PostgreSQL rows.

> **Security boundary:** The GitHub Pages frontend is public by design. The public Supabase key identifies the project but does not authorize unrestricted writes. Row-level security and least-privilege grants are the access controls; a service-role or secret key must never be included in the browser application.

7. Implementation

7.1 Architecture

The project has a static frontend and managed backend services. GitHub Pages serves the HTML pages, CSS, JavaScript modules, and assets. The Supabase JavaScript client runs in the browser and calls the Supabase Data API for PostgreSQL operations. Supabase Auth manages the session. PostgreSQL stores relational records and applies constraints and row-level security. Supabase Storage stores uploaded image files.

Layer	Technology	Responsibility
Presentation	HTML, CSS, JavaScript	Forms, navigation, cards, filters, dashboard, validation messages.
Hosting	GitHub Pages	Serves static files over the project site URL.
Client data access	`@supabase/supabase-js`	Reads and writes exposed rows using the active session and public project key.
Identity	Supabase Auth	Email/password account flows; auth user ID is linked to `app_users`.
Database	Supabase PostgreSQL	Stores application rows, validates constraints, executes triggers and RLS policies.
File storage	Supabase Storage	Stores item photos in the `item-images` bucket and returns a public URL.

7.2 Schema creation and initialization

The schema is defined in `db/migrations/001_initial_schema.sql`. It enables `pgcrypto`, `citext`, and `pg_trgm`, defines PostgreSQL enum types, creates the relations and indexes, seeds roles and categories, adds timestamp/profile synchronization triggers, enables RLS on the current exposed tables, defines policies and grants, and creates the public `item-images` storage bucket with an image size limit and allowed MIME types. The repository README records the Supabase setup and GitHub Pages deployment steps.

7.3 Representative SQL

The following query is a relational equivalent of the listing data the frontend displays: it joins an item to its category and optional location, filters to lost reports, and orders newest first.

```
SELECT i.id, i.title, i.kind, i.status,

       c.name AS category,

       l.name AS location,

       i.created_at
```

```
FROM public.items AS i

JOIN public.categories AS c ON c.id = i.category_id

LEFT JOIN public.locations AS l ON l.id = i.location_id

WHERE i.kind = 'lost'

ORDER BY i.created_at DESC;
```

The migration uses a profile lookup in its owner policies. This representative policy allows an authenticated user to insert an item only when its owner profile maps to the caller's authenticated user ID.

```
CREATE POLICY "users create their own items"

ON public.items FOR INSERT TO authenticated

WITH CHECK (

  owner_id = (

    SELECT id FROM public.app_users

    WHERE supabase_auth_user_id = (SELECT auth.uid())

  )

);
```

7.4 Application CRUD flow

- **Create:** The client resolves the selected category and signed-in profile, inserts a location if supplied, inserts the item row, then uploads an optional image and inserts its metadata. If the image metadata step fails after upload, the client attempts to remove the uploaded file.
- **Read:** Public pages query items by kind and request related category, location, owner reference, and image metadata. Search and category/status sorting are currently performed on the loaded results in the browser.
- **Update:** The dashboard submits changed item fields or status. A database trigger sets `updated_at`; RLS limits the update to the item's owner.
- **Delete:** The dashboard deletes an owned item and removes the associated image file. Database foreign-key actions cascade dependent image metadata.

8. Testing and Validation

The following read-only checks were carried out against the deployed public site on 12 October 2026 while preparing this report. They confirm the listing and filtering behaviors visible to a visitor. The record counts are a snapshot and can change as demo data changes.

Test	Input/action	Expected	Observed result	Status
TC-01	Open the Lost Items page.	Page loads and displays lost listings.	Two cards displayed: “Demo Item” and “DEMO QA Lost Backpack”.	PASS
TC-02	Search for `DEMO QA`.	Only matching listing remains.	One card displayed: “DEMO QA Lost Backpack”.	PASS
TC-03	Clear search and filter status to Claimed.	Only claimed lost reports remain.	One card displayed: “Demo Item” with Claimed status.	PASS
TC-04	Search for a string not present in the listings.	Zero cards and a clear no-results state.	Count showed 0 items; the page displayed “No items match your search.”	PASS
TC-05	Open the Found Items page.	Found listings load separately from lost listings.	One found card displayed: “Lost Backpack”.	PASS

8.1 Integrity and access-control validation plan

These checks are not marked as passed because they were not executed against the live database during this report preparation. They should be performed in a staging project or with reversible demo records before final assessment.

Test	Setup/input	Expected database behavior	Execution status
TC-06	Submit an item with a blank title or description.	Frontend validation should block submission; database checks also reject blank/oversized text.	Not exercised here
TC-07	Attempt to insert an item using a nonexistent owner or category ID.	Foreign-key constraint rejects the orphan record.	Not exercised here
TC-08	As one authenticated user, update or delete a different user's item.	RLS prevents the operation.	Policy present; not exercised here
TC-09	Create, edit, and delete an item through an authenticated account.	Only the account's own record changes; related image metadata stays consistent.	Not exercised here
TC-10	Upload a non-image or oversized file.	Storage bucket MIME-type and size settings reject the file.	Not exercised here

Test	Setup/input	Expected database behavior	Execution status
TC-11	Submit a valid contact form and inspect its database row.	A row is created in `contact_messages`.	Not exercised here

The source migration contains the listed constraints and policies, but their presence in code is not the same as a live negative test. No write operations were performed while collecting the live screenshot evidence.

9. Results and Screenshots

The deployed application is accessible at https://adithyanandanarun.github.io/Lost-and-Found/. The live home page loaded in a signed-in demo state. The home-page and Lost Items screenshots in Section 4 show the account navigation and the existing “Demo Item” report alongside another demo listing. The Found Items page also loaded a separate listing. Searching, status filtering, and the empty-results message produced the expected visible results during the read-only checks.

Evidence	Result shown
Home screen capture	Shows the application entry points and signed-in navigation controls.
Lost Items screen capture	Shows the existing demo lost item, another listing, and the search/filter controls.
ER diagram	Shows the PostgreSQL relations and their key relationships, including the schema groundwork for future modules.
Read-only checks TC-01 to TC-05	Listing retrieval, text search, status filtering, empty-state handling, and the Found Items page all produced the expected visible result.

The screenshots were captured from the deployed site and the ER diagram file in the repository's db folder. Live item counts and statuses can change when demo records are updated.

10. Challenges Faced and Solutions

Challenge	Response in the project	Remaining consideration
Connecting a public static frontend to database services without a private server.	The frontend uses the Supabase JavaScript client with a public project key. RLS policies and grants enforce what anonymous and authenticated visitors may read or change.	Keep service-role/secret keys out of the repository and review RLS whenever a table or policy changes.
Maintaining consistent links between an item, its owner, category, location, and image.	Separate lookup/profile tables and foreign keys represent these relationships. Database constraints prevent invalid references.	Locations are currently inserted from the form when supplied; a future version could reuse existing locations to avoid duplicates.
Handling optional image upload in a multi-step post operation.	Image bytes are stored in Supabase Storage and metadata is inserted in PostgreSQL. If metadata insertion fails after upload, the client attempts cleanup of the uploaded object.	More robust retry/transaction coordination and upload error feedback can be added.
Email provider rate limits during demo account setup and password reset.	The current walkthrough can use the already signed-in demo account instead of repeatedly creating accounts or requesting reset emails.	For dependable email flows, configure a custom SMTP provider and test confirmation/reset redirects.
Moving from an earlier Firebase-based prototype to PostgreSQL.	The current application now reads and writes to Supabase; the repository includes a migration script for Firestore records.	The README states that legacy Firebase accounts, posts, and images have not been transferred. A migration should be planned and validated before relying on old data.
Search behavior versus larger data volumes.	The current interface filters fetched results in the browser, while PostgreSQL trigram indexes are included in the schema.	Move filtering and pagination into database queries as the number of reports grows so the indexes can be used by the live listing query.

11. Conclusion and Future Enhancements

The project delivers a web application for publishing and browsing lost and found reports. The deployed static frontend provides separate listing pages, search and filters, user authentication, post management, image upload, a dashboard, and a contact form. Supabase PostgreSQL stores the core application data and connects items to profiles, categories, locations, and image metadata. Constraints and row-level security policies provide database-level integrity and owner-scoped access. The ER model is converted into normalized relations, and the current live demo demonstrates listing retrieval and read-only filtering.

The project also establishes schema foundations beyond the current interface. Claim and evidence records, conversations, messages, notifications, tags, role assignments, and item status history can support later features but are not yet implemented in the web app. The read-only report checks passed for listing display, search, status filtering, no-result handling, and the found-item page. Authenticated write operations and negative security/constraint cases remain to be exercised as dedicated tests.

11.1 Future enhancements

- Implement a claim workflow with claimant evidence, owner/moderator review, and status transitions.
- Add private conversations and notifications related to claims.
- Write status-history entries automatically and expose a post history view.
- Move search, category/status filters, sorting, and pagination to database-side queries.
- Add moderation and role-management screens, with policies for all claim/messaging tables before exposing them.
- Configure custom SMTP and thoroughly test sign-up, confirmation, sign-in, and password-reset flows.
- Plan and validate migration of legacy Firebase accounts, items, and images if that data is needed.
- Add automated integration tests for row-level security, foreign-key/check constraints, image permissions, and CRUD behavior.

References

1. Lost and Found Web App source repository. *README.md*, *db/README.md*, and *db/migrations/001_initial_schema.sql*. Project implementation and schema files.
2. PostgreSQL Global Development Group. *PostgreSQL 18 Documentation: Constraints*. https://www.postgresql.org/docs/current/ddl-constraints.html (accessed 12 October 2026).
3. PostgreSQL Global Development Group. *PostgreSQL 18 Documentation: Index Types*. https://www.postgresql.org/docs/current/indexes-types.html (accessed 12 October 2026).
4. Supabase. *Database Overview*. https://supabase.com/docs/guides/database/overview (accessed 12 October 2026).
5. Supabase. *Securing your data*. https://supabase.com/docs/guides/database/secure-data (accessed 12 October 2026).
6. Supabase. *Row Level Security*. https://supabase.com/docs/guides/database/postgres/row-level-security (accessed 12 October 2026).
7. Supabase. *Auth*. https://supabase.com/docs/guides/auth (accessed 12 October 2026).
8. Supabase. *Storage Access Control*. https://supabase.com/docs/guides/storage/security/access-control (accessed 12 October 2026).
9. Supabase. *JavaScript: Initializing*. https://supabase.com/docs/reference/javascript/initializing (accessed 12 October 2026).
10. GitHub Docs. *What is GitHub Pages?*. https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages (accessed 12 October 2026).